

User Guide

codeAnalyzer reads a folder of source code on your computer and shows you clear, per-file and per-method statistics. **Everything runs on your own machine** — nothing is uploaded, and there is no internet connection involved.

This guide shows you how to run it and what every number on the screen means.

1. Starting the app

1. Open the codeanalyzer folder you were given and double-click **codeanalyzer.exe**.
2. A small black window opens (leave it open — that's the program running) and your web browser opens automatically at an address like `http://127.0.0.1:5000`.
3. When you're done, close the browser tab and close that black window to stop the app.

If the browser doesn't open by itself, copy the address shown in the black window and paste it into your browser.

2. Running an analysis

You'll see a simple form with two boxes:

- **Folder path** — the folder you want to analyze. Click **Browse...** to pick it, or type the full path (e.g. `C:\Projects\MyApp\src`). The app looks through that folder **and all sub-folders**.
- **Prefix** — the naming prefix that marks the methods you care about (default `Tcf`). Any C# method whose name *starts with this prefix* is analyzed in detail. Change it if your team uses a different prefix.

Click **Analyze**. A short "Analyzing..." screen appears, then the results. The app remembers your last folder and prefix for next time.

Which files are read: C# (`.cs`) and C/C++ (`.c .h .cpp .hpp .cc .cxx .hh`). Other files are ignored.

3. Understanding the results

Results are shown in five sections, top to bottom.

Project Summary

The big picture — totals for the whole folder: number of files, total lines, and how many of them are code / comments / blank, plus method counts. A quick health snapshot.

File Summary

One row per file. Click any column heading to **sort**; type in the box to **filter**. Columns:

Column	Meaning
Total / Code / Cmt / Inl / Blank	line counts for that file (see the glossary, §5)
Type	the language (C#, C++, header, ...)
Prefix-method count (e.g. <i>TCF</i>)	how many prefix methods are defined in that file
Helpers	how many helper functions are defined in that file

C and C++ files get line counts only; their method/helper columns are always 0.

Source Tree

A collapsible folder/file tree. Click a file to expand it and see the prefix methods inside; click a method to jump straight to its details. Use **Expand all** / **Collapse all** to open or close everything at once.

Method Details

One card per prefix method. Each card shows:

- the method **name** and its **file and line range**;
- Cyclomatic Complexity** — how many decision paths the method has. Higher = harder to test and maintain. The badge is **neutral up to 10, amber above 10, red above 20**.
- a **comment ratio** bar (turns red when a sizeable method has very few comments);
- "calls N helper(s)"** — the project helper functions this method uses, listed as chips below.
"Helpers" here means *other functions this method calls*, not functions defined inside it.

Use the **search box** to find a method, the **Per page** selector to show more or fewer at once, and the arrows to move between pages.

Helper Usage Summary

The reverse view: each **helper function** and exactly **which methods call it**. Useful for seeing how widely a piece of shared code is used.

4. Saving the results

In the top bar:

- **Download .txt** — the full report as plain text.
- **Download .json** — the full report as structured data (for feeding into other tools).

Each table also has a **CSV** button to export just that table (opens in Excel). All exports are created on your machine; nothing is sent anywhere.

5. Glossary — what the words mean

- **Prefix method (e.g. TCF method)** — a C# method whose name starts with the configured prefix. These are the methods analyzed in detail.
 - **Helper function** — a method in your own code (not a built-in/library method) that at least one prefix method calls. A function nobody calls is *not* counted as a helper, and built-in calls (like `List.Add`) are never counted as helpers.
 - **Total lines** — every line in the file.
 - **Code lines** — lines containing actual code.
 - **Comment lines** — lines containing a comment.
 - **Inline comments** — lines that have *both* code and a comment (e.g. `x = 5; // note`). Such a line counts once as code and once as a comment.
 - **Blank lines** — empty lines.
 - **Comment ratio** — comments ÷ total lines; a rough readability indicator.
 - **Cyclomatic complexity** — a count of the independent paths through a method (`if`, loops, `&&`, `case`, etc.). Lower is simpler.
-

6. Messages and warnings

The app won't crash on bad input — it tells you instead:

- **Folder doesn't exist / isn't a folder** — check the path you typed.
 - **No supported source files found** — the folder has none of the file types above.
 - **Warnings panel** (yellow) — e.g. a file with a syntax error or an unusual text encoding. The rest of the files are still analyzed normally; only the flagged file may be incomplete.
-

7. Privacy

codeAnalyzer is **local-only by design**: it makes no internet requests, uses no cloud or AI services, and never sends your source code or results off the machine. The little website it shows is served only to your own computer (`127.0.0.1`). The address includes a one-time access code so nobody else on a shared machine can open your session.